



Моделювання UML

к.т.н. Мирослава ГЛАДКА

Що таке UML-діаграми?

- Unified Modeling Language (UML) — уніфікована мова моделювання. Розшифруємо: modeling передбачає створення моделі, що описує об'єкт. Unified (універсальний, єдиний) — підходить для широкого класу проєктованих програмних систем, різних областей додатків, типів організацій, рівнів компетентності, розмірів проєктів.
- UML описує об'єкт в єдиному заданому синтаксисі, тому де б ви не намалювали діаграму, її правила будуть зрозумілими для всіх, хто знайомий з цією графічною мовою

Для чого використовується UML?

- Одне із завдань UML — служити засобом комунікації всередині команди та при спілкуванні з замовником.

Можливі варіанти використання діаграм.

- Проектування. UML-діаграми стануть у пригоді при моделюванні архітектури великих проектів, в якій можна зібрати як великі, так і дрібніші деталі і намалювати каркас (схему) програми. По ньому пізніше буде будуватись код.
- Реверс-інжиніринг — створення UML-моделі з існуючого коду додатку, зворотна побудова. Може застосовуватися, наприклад, на проектах підтримки, де є написаний код, але документація неповна або відсутня.
- З моделей можна витягувати текстову інформацію і генерувати відносно читабельні тексти — документувати. Текст і графіка будуть доповнювати один одного.

UML

- Основними характеристиками об'єктно орієнтованої мови моделювання UML є:
- організація взаємодії замовника і розробника (груп розробників) ІС шляхом побудови репрезентативних візуальних моделей;
- спеціалізація базових позначень для конкретної предметної області

Нотація UML для опису логіки проекту.

- Як будь-яка інша мова, UML має власні правила оформлення моделей і синтаксис. За допомогою графічної нотації UML можна візуалізувати систему, об'єднати всі компоненти в єдину структуру, уточнювати і покращувати модель у процесі роботи.

На загальному рівні графічна нотація UML містить 4 основні типи елементів:

- фігури;
 - значки;
 - лінії;
 - написи.
- UML-нотація де-факто є галузевим стандартом у сфері розробки програмного забезпечення, ІТ-інфраструктури і бізнес-систем.

Види UML-діаграм

В мові UML є 12 типів діаграм:

- 4 типи діаграм представляють статичну структуру додатку;
- 5 типів представляють поведінкові аспекти системи;
- 3 представляють фізичні аспекти функціонування системи (діаграми реалізації).

Найбільш доступними з них є:

- Діаграма прецедентів (Use-case diagram);
- Діаграма класів (Class diagram);
- Діаграма активностей (Activity diagram);
- Діаграма послідовності (Sequence diagram);
- Діаграма розгортання (Deployment diagram);
- Діаграма співробітництва (Collaboration diagram)
- Діаграма об'єктів (Object diagram)
- Діаграма станів (Statechart diagram).

У нотації мови UML визначені наступні види діаграм

- варіантів використання (use case diagram);
- класів (class diagram);
- кооперації (collaboration diagram);
- послідовності (sequence diagram);
- станів (statechart diagram);
- діяльності (activity diagram);
- компонентів (component diagram);
- розгортки (deployment diagram);

Класифікація груп діаграм UML 2.0

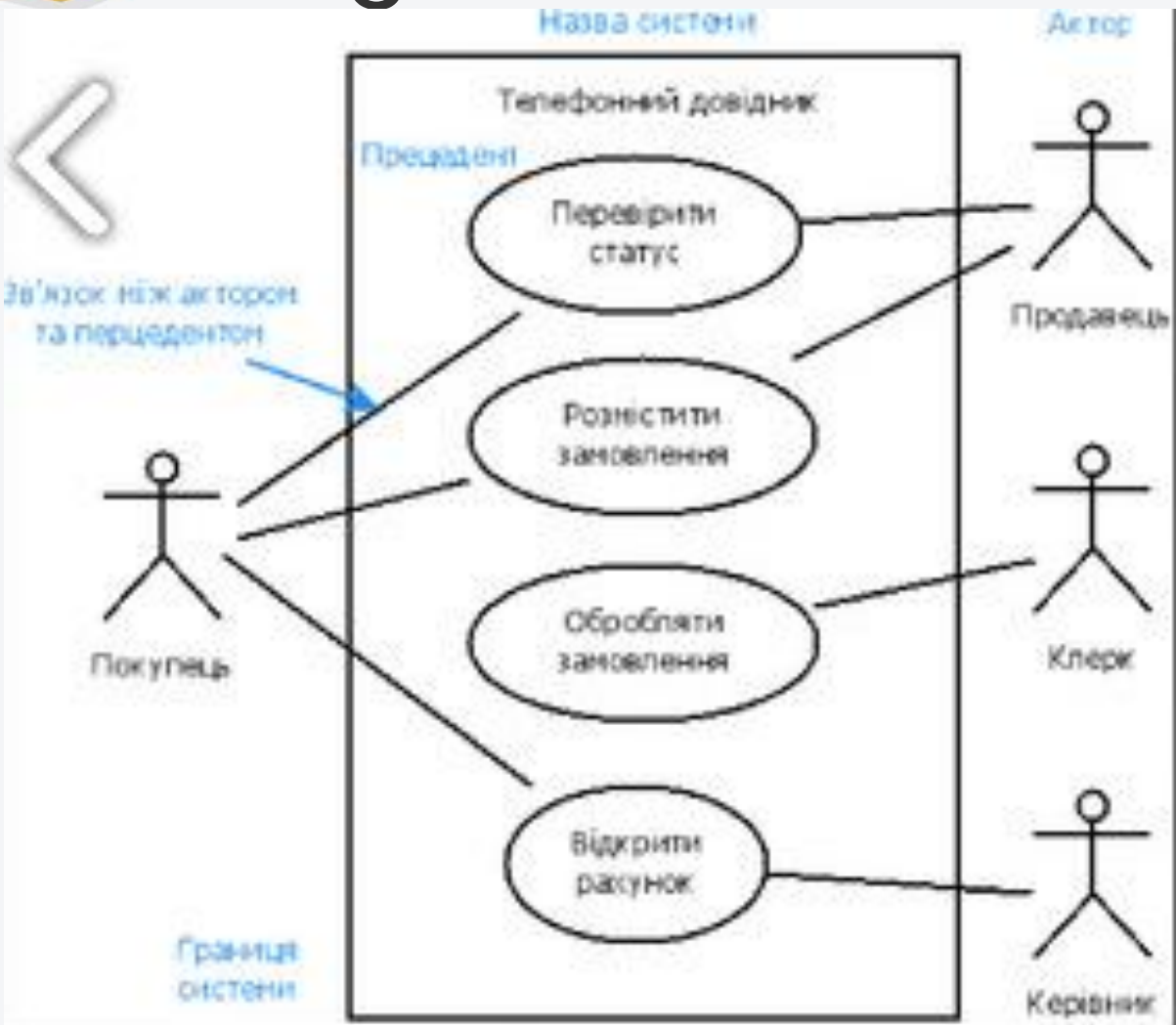


Діаграма прецедентів — Use-case diagram.

Основними елементами для побудови моделі прецедентів на діаграмі є

- Актор (actor) – елемент, що позначає ролі користувача, який взаємодіє з певною сутністю;
- Прецедент – елемент, що відображає дії, що виконуються системою (в т.ч. із зазначенням можливих варіантів), які призводять до результатів, спостережуваним акторами.

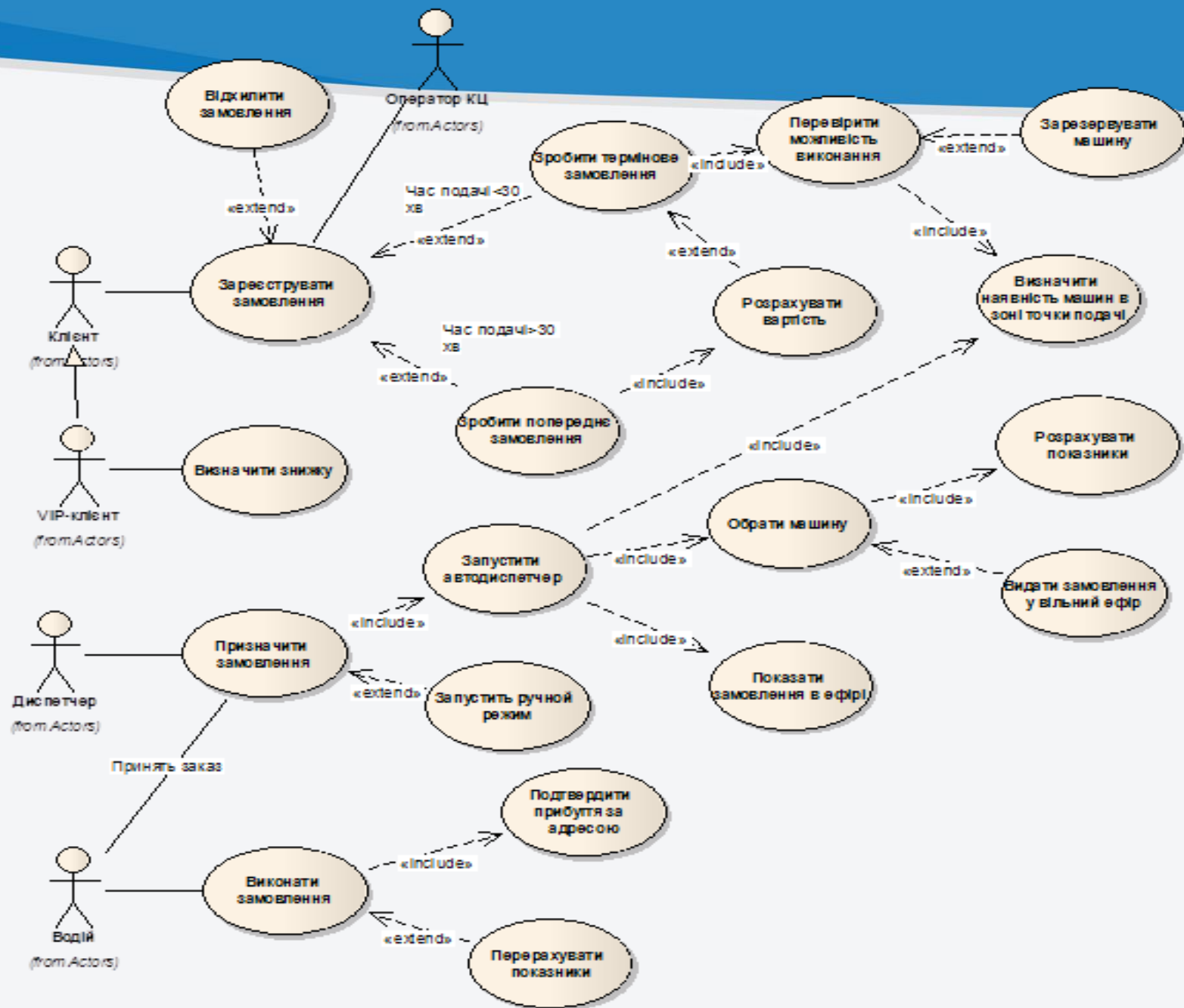
Діаграма прецедентів — Use-case diagram.



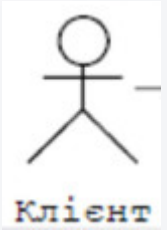
Діаграма прецедентів — в UML, діаграма, на якій зображено відношення між акторами та прецедентами в системі. Також, перекладається як діаграма варіантів використання. Суть даної діаграми полягає в наступному: проектувана система представляється у вигляді безлічі сутностей чи акторів, що взаємодіють із системою за допомогою так званих варіантів використання. Діаграми прецедентів відображають елементи моделі варіантів використання.

Приклад діаграми прецедентів для інформаційної системи Оформлення системи

ис Оформлення замовлення



Актори (actors) діаграми прецедентів



- Важливо зауважити два моменти: Поняття актора несе ролеву ознаку. (Актори, по суті, – це типи: актором є клієнт чи адміністратор, а не Петренко, Шевченко чи Байден).
- Акторами можуть виступати інші програмні системи. Актори позначаються у діаграмах прецедентів стилізованими людськими фігурками. Приклади акторів: Клієнт, адміністратор, білінгова система.

Як визначати акторів?

Простіше спочатку визначити основних акторів.

- Основні актори – це ті актори, які ініціюють взаємодію з програмною системою.
- Для визначення основних акторів необхідно розглянути типові ситуації використання проектованої системи, і типовими користувачами системи й будуть основні актори.
- Далі, розглядаючи відповідальності основних акторів та проектованої системи, можна визначити другорядних актор

Приклади другорядних акторів:

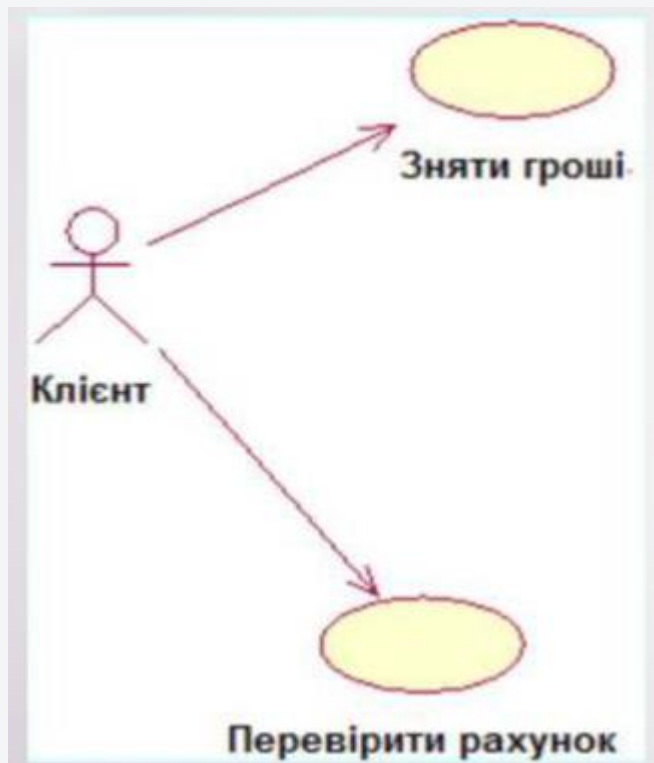
- інша (наприклад, білінгова) система,
- експерт з перевірки дорожніх інцидентів (у страховій фірмі).

Прецеденти (Use case)



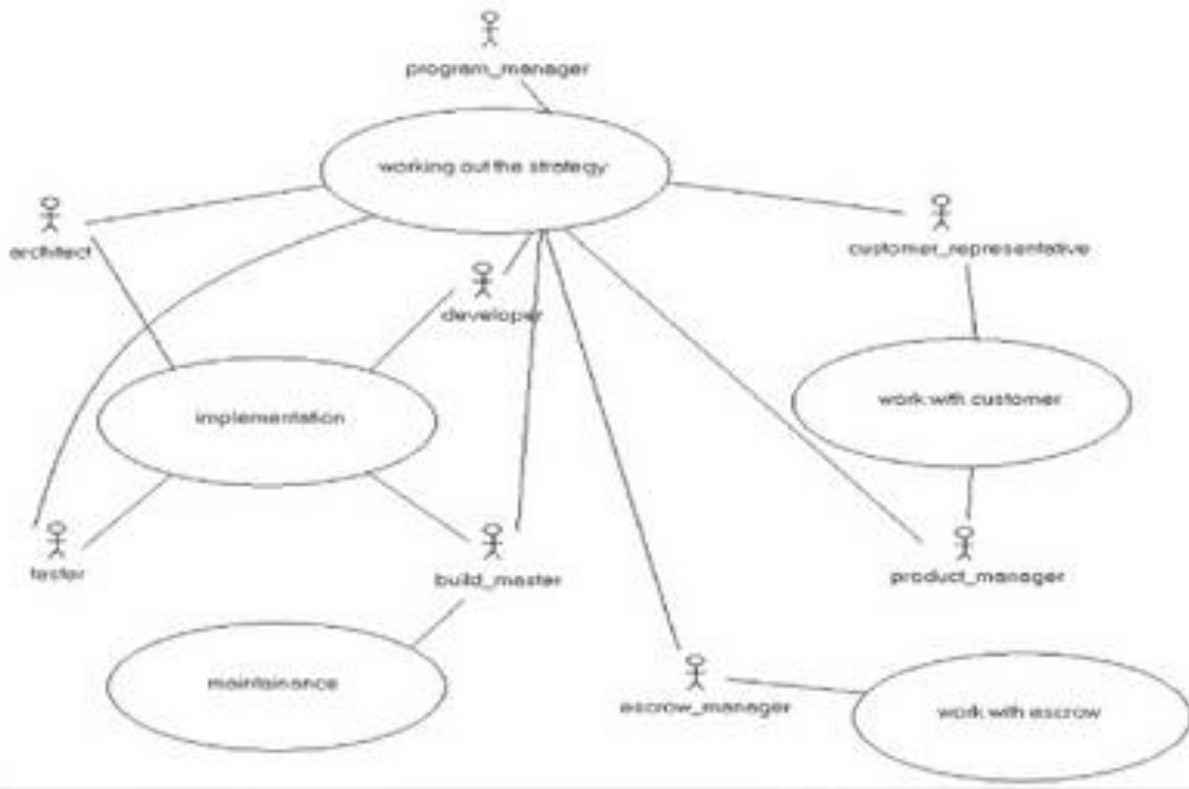
- Як визначати прецеденти?
- Прецеденти зображуються еліпсами.
- Як правило, для іменування прецедентів використовуються дієслова чи короткі дієслівні фрази ("Створення таблиці", "Створити таблицю")
- Простий спосіб виявлення прецедентів полягає в тому, що треба розглянути кожного основного актора та визначити, які суттєві результати (суттєві цілі) він може переслідувати.
- Отже, прецеденти мають функціональне забарвлення, "прив'язуючись" до суттєвих цілей основних акторів.

Прецеденти. Приклад (банкомат)



- Прецеденти дозволяють специфікувати функціональну поведінку розроблюваної системи та отримати відповідь на запитання, що має робити системи, проте не визначають реалізацію цієї поведінки системи – не торкаються питань, яким чином відповідні функції реалізуються.

Прецеденти. Приклад



- Бачимо, що одна і та сама особа може виступати в декількох ролях. Наприклад, product manager у нас працює над стратегією і більше нічим не займається, архітектор працює над стратегією і займається впровадженням, build master займається трьома речами одночасно, і так далі. За такою схемою ми можемо простежити, як пов'язані між собою ролі і прецеденти.

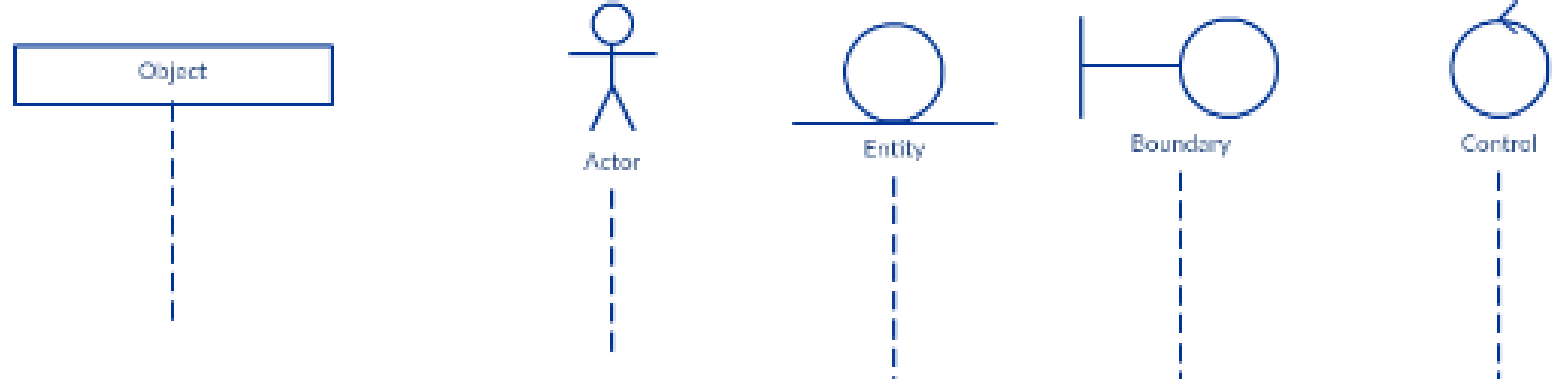
Діаграма послідовності

- Діаграма послідовності (Sequence Diagram)- різновид діаграми в UML. Відображає взаємодії об'єктів впорядкованих за часом. Зокрема, такі діаграми відображають задіяні об'єкти та послідовність відправлених повідомлень.

Діаграма послідовності

- Діаграми послідовностей можна використовувати для уточнення діаграм прецедентів, більш детального опису логіки сценаріїв використання.
- Діаграми послідовностей зазвичай містять об'єкти, які взаємодіють у рамках сценарію, повідомлення, якими вони обмінюються, і які повертають результати, які пов'язані з повідомленнями.
- На діаграмі послідовності зображуються тільки ті об'єкти, які безпосередньо беруть участь у взаємодії.

Діаграма послідовності



- **Об'єкт (учасник)** — позначення лінії життя та екземпляр класу у горизонтального прямокутника.
Актор — використовується, коли конкретна діаграма послідовності належить варіанту використання.
Сутність (entity) — представляє системні дані. Наприклад, у програмі обслуговування клієнтів суб'єкт — клієнт керує даними (сутність), пов'язаними з клієнтом.
Межа/кордон (boundary) — вказує на межу системи/ граничний елемент у системі; наприклад, екрани інтерфейсу користувача, шлюзи баз даних або меню, з якими взаємодіють користувачі
Управління (control) вказує на керівну сутність або менеджера. Він організовує та планує взаємодії між кордонами та сутностями та служить посередником між ними

Діаграма послідовності

- На діаграмах послідовностей об'єкти позначено прямокутниками з підкресленими іменами
- Повідомлення — лініями зі стрілками.
- Суцільними лініями показано виклики методів
- Пунктирними — результати відпрацювання методів

учень1:Абітурієнт



Вступ учня до вишу



- На основі діаграми прецедентів вступу випускників до закладу вищої освіти створимо діаграму послідовностей.

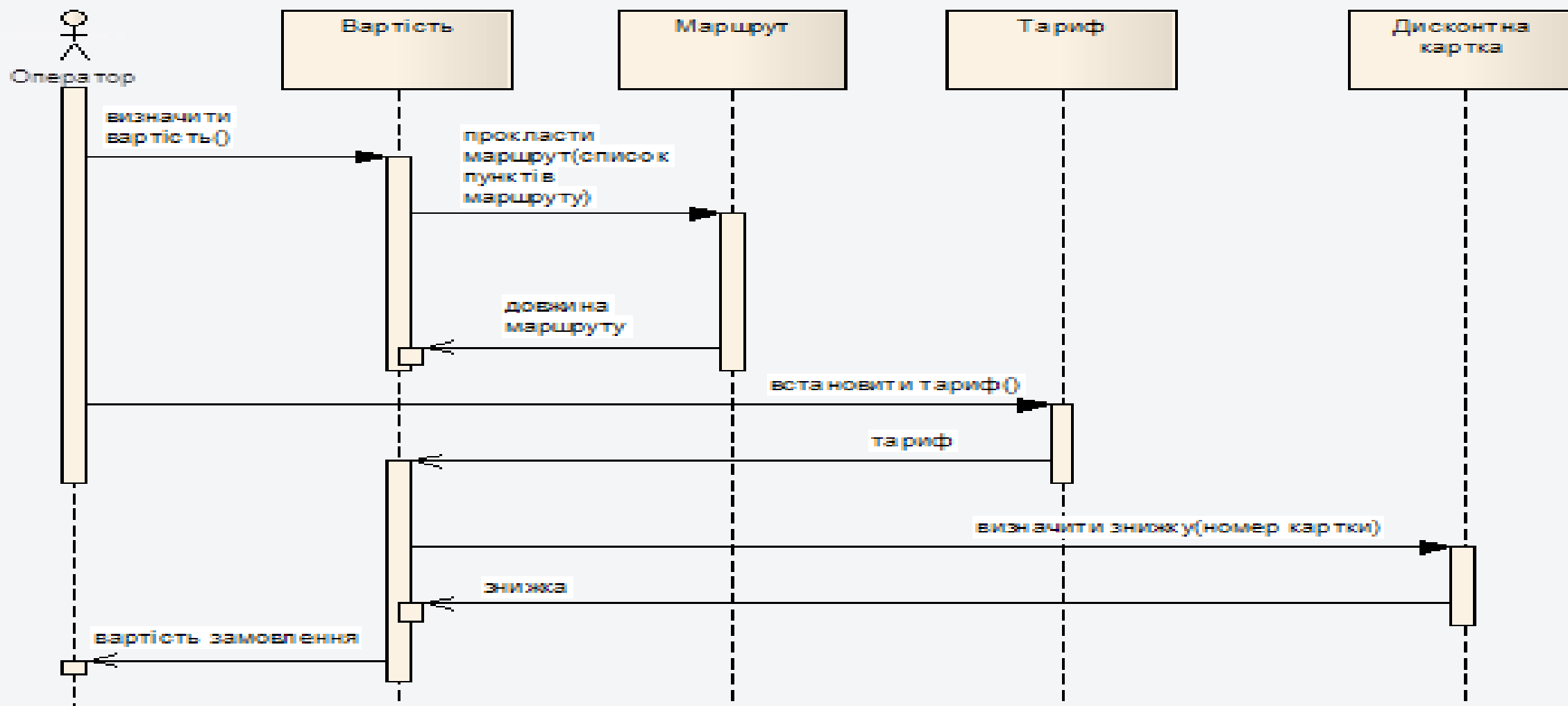
- Зміст діаграми у словесній формі можна подати так. Учень хоче стати студентом певного факультету. З цією метою він подає заяву в приймальну комісію, а та приймає від нього заяву, учень змінює стан на абітурієнта, конкурсна комісія перевіряє кількість балів і запитує результати ЗНО, приймає, визначає рейтинг учня серед абітурієнтів факультету, передає їх до приймальної комісії. Учень отримує результат з приймальної комісії.

Діаграма послідовності

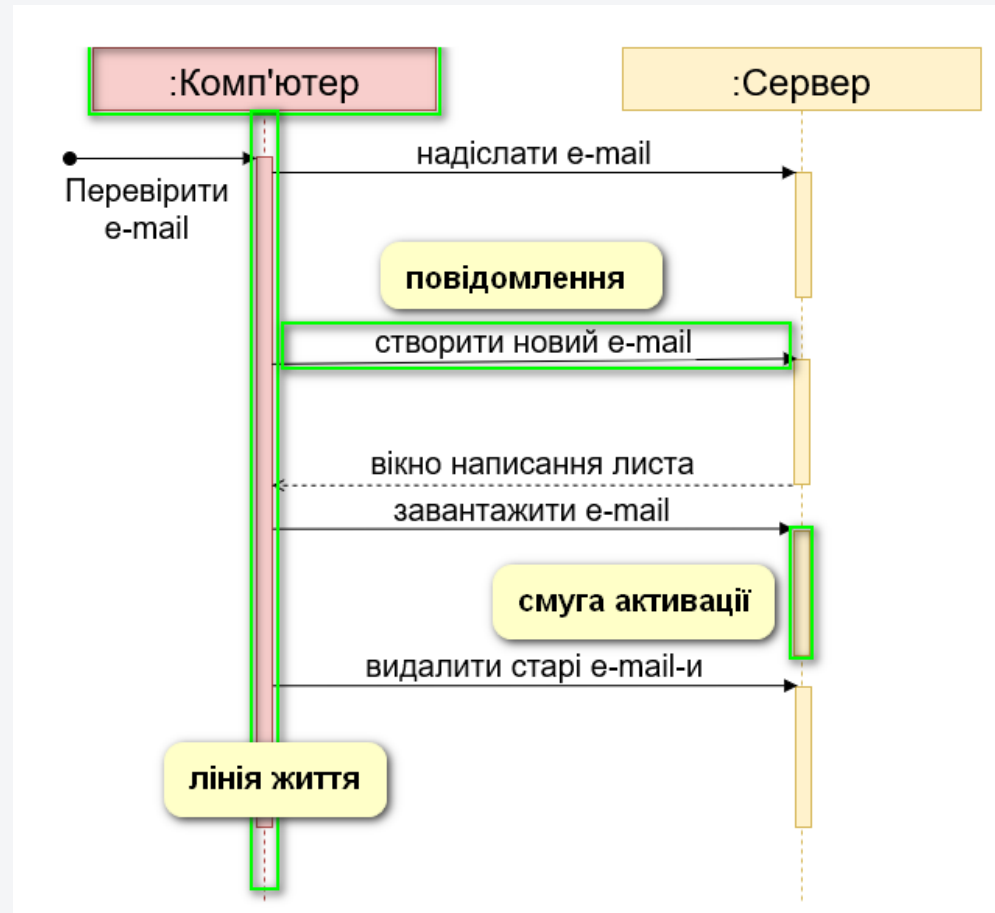


Приклад діаграми послідовності

sd DGP S



Діаграма послідовності

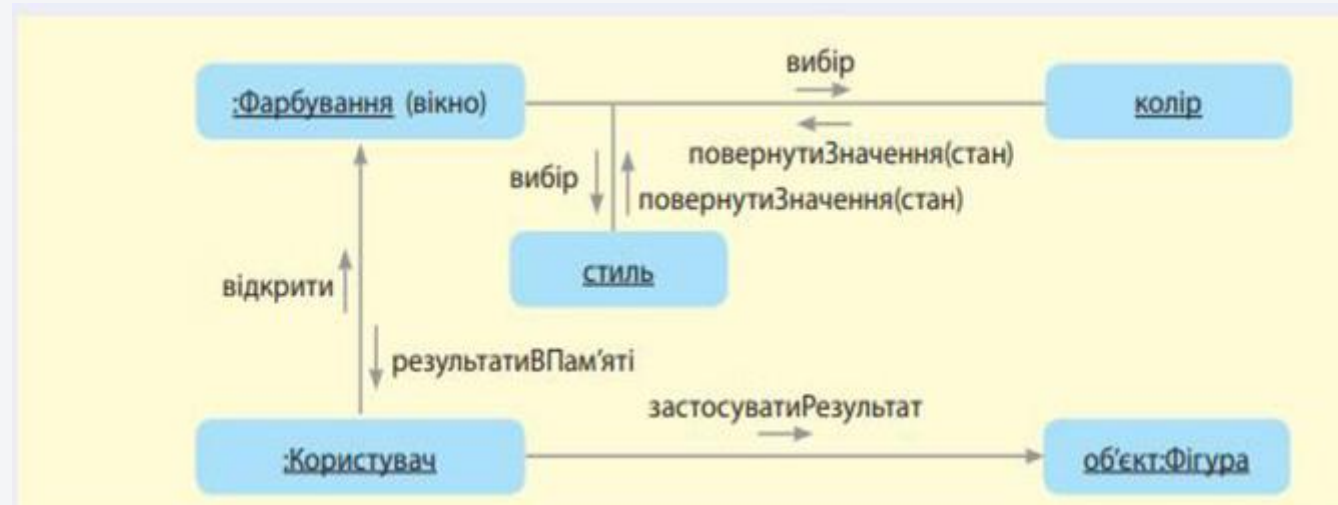


Діаграма взаємодій

- Альтернативою діаграми послідовностей є діаграма взаємодій.
- На ній зазначається не назва операції, а її порядковий номер.
- Оскільки від одного об'єкта може викликатися кілька методів, а між викликами отримувались результати, то в розташуванні номерів за часом не буде впорядкованості.

Діаграма взаємодій

- На рисунку подано заготовку діаграми взаємодій: показано, як у процесі створення графічного зображення користувач зафарбовує фігуру. Замість методів і їх результатів у діаграмі взаємодій проставляється нумерація в послідовності виконання.



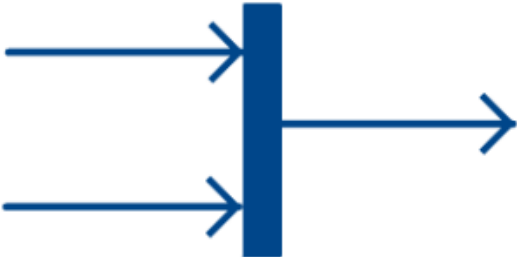
Діаграма діяльності

- Діаграма діяльності (англ. activity diagram) — в UML та SysML, візуальне представлення графу діяльностей. Граф діяльностей є різновидом графу станів скінченного автомату, вершинами якого є певні дії, а переходи відбуваються по завершенню дій.

Позначення діаграми діяльності

Символ	Ім'я	Використання
●	Початковий вузол	Відправна точка, або початковий стан
	Дія	Представлення діяльності, завдання для виконання
	Потік керування	Спрямований потік, контрольний потік діяльності
	Кінцевий вузол активності	Кінцевий стан, завершення усіх потоків процесу
	Кінцевий вузол потоку	Кінець одного потоку
	Вузол прийняття рішення	Розгалуження з умовою та кількома варіантами дій. Має один вхід декілька виходів

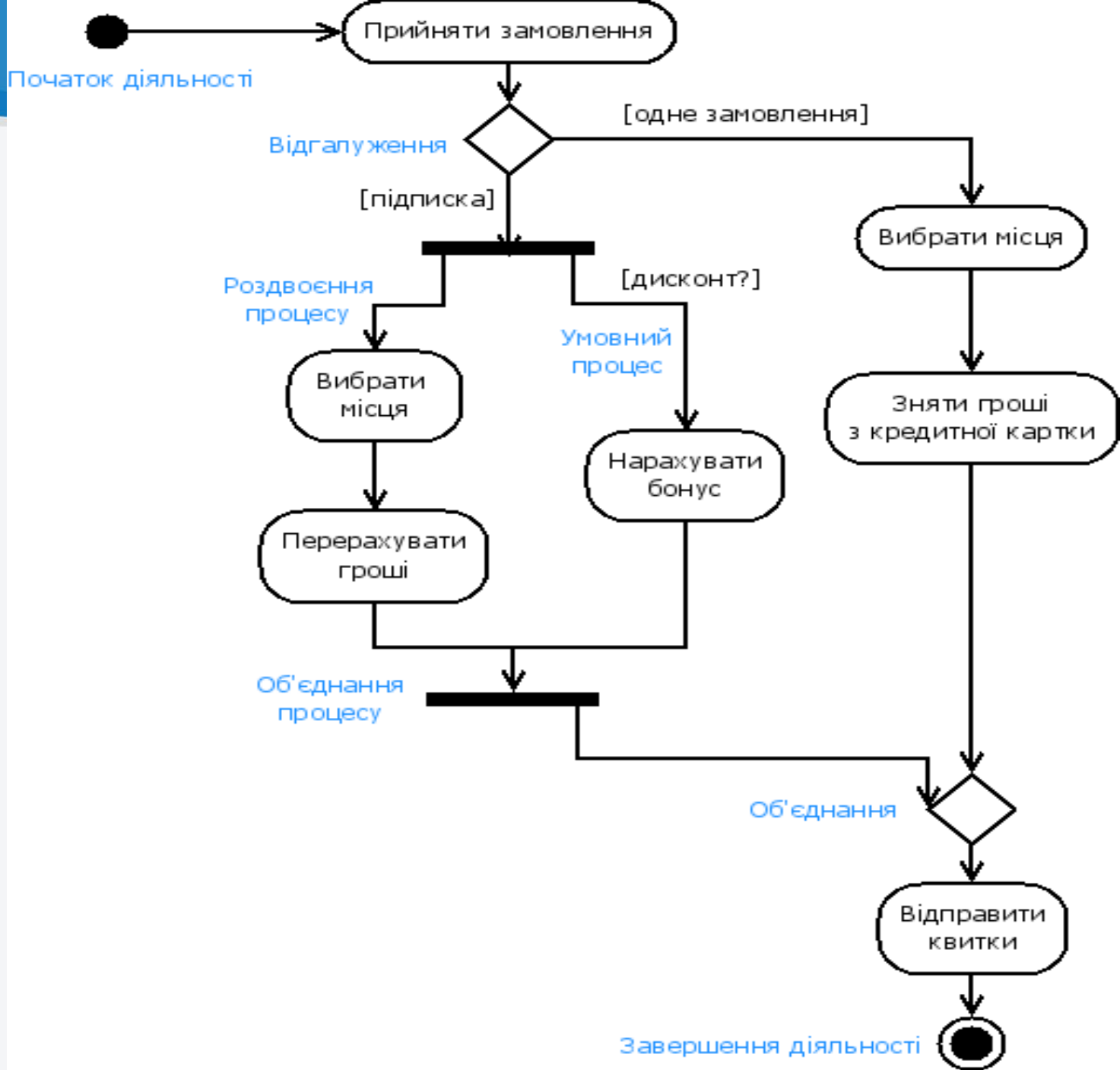
Позначення діаграми діяльності

	Вузол злиття	Об'єднання потоків створених вузлом прийняття рішень. Має кілька входів і один вихід
	Вилка	Розподілення потоку на кілька паралельних без прийняття рішення
	Злиття	Об'єднання декількох паралельних потоків

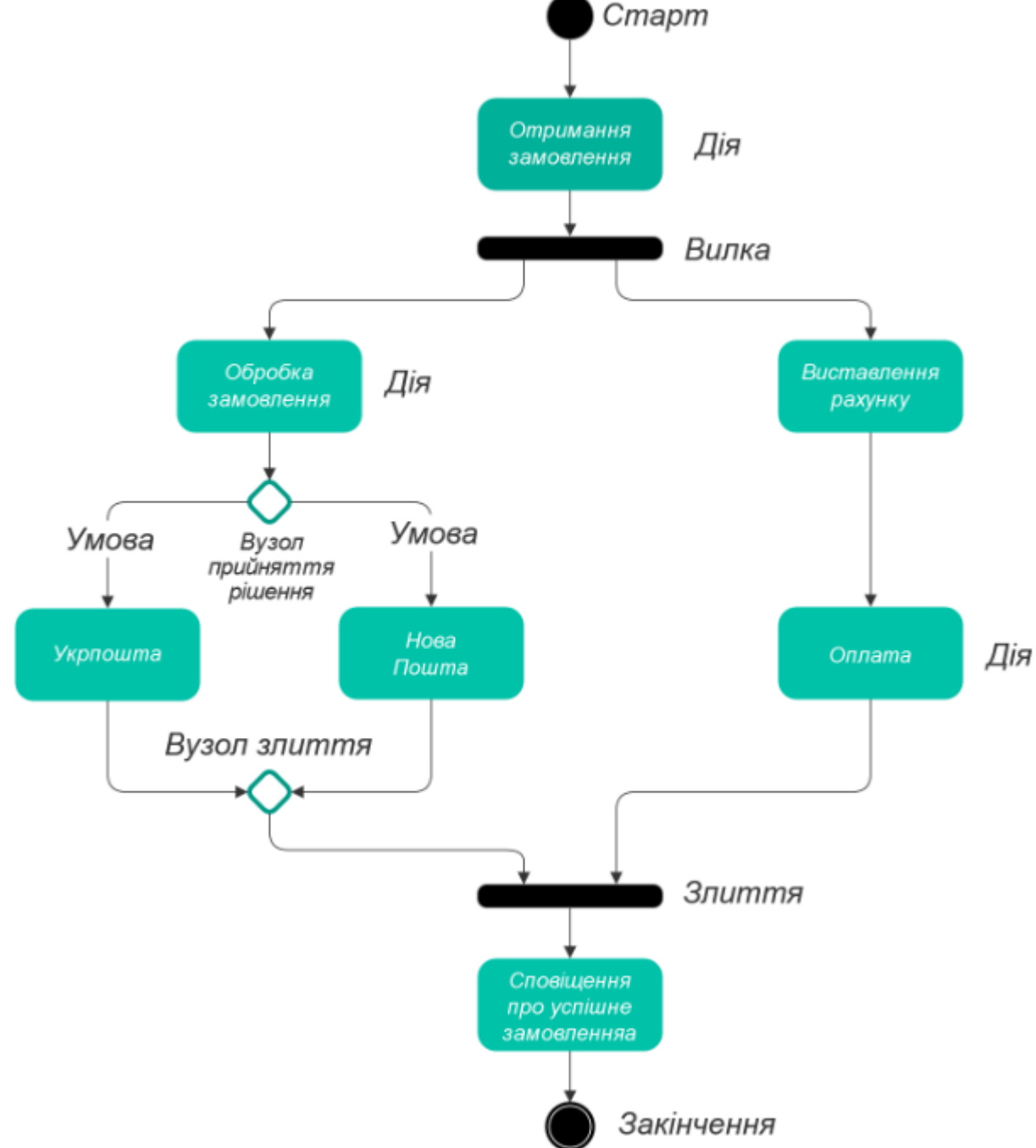
Позначення діаграми діяльності

	Надсилання сигналу	Вказує на те що сигнал надсилається на приймальну діяльність
	Отримання сигналу	Вказує на отримання сигналу
	Коментар	Дозволяє робити коментарі до діаграми. З'єднується пунктирною лінією

Приклад діаграми діяльності



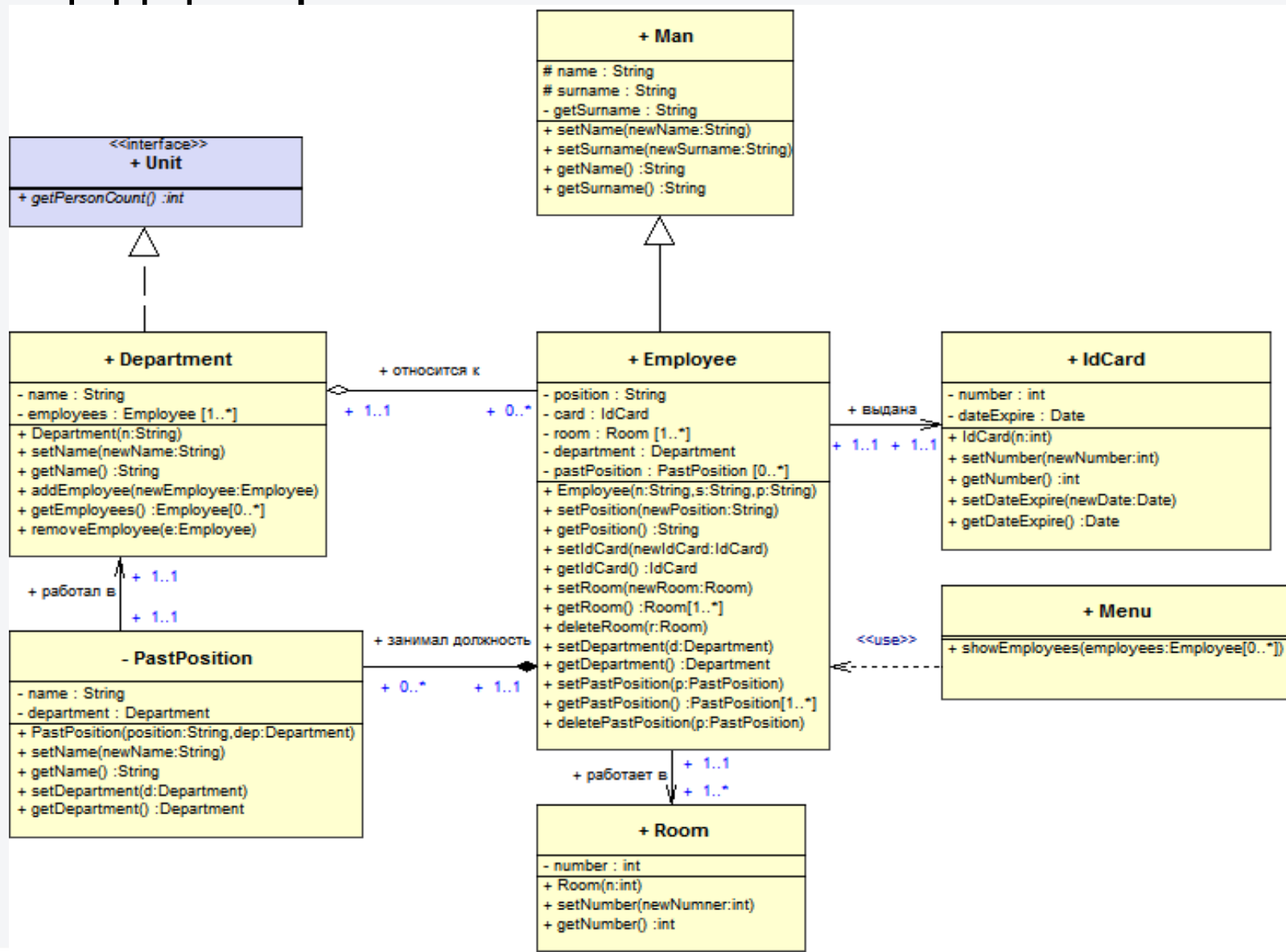
Приклад діаграми діяльності



Діаграма класів (class diagram)

- Діаграма класів (class diagram) — статичне представлення структури моделі.
- Відображає статичні елементи, такі як: **класи, типи даних, їх зміст та відношення.**
- Діаграма класів є ключовим елементом редактора UML-діаграм, оскільки часто додатки генеруються саме з діаграми класів Діаграма класів (class diagram)
- На відміну від діаграми послідовностей, діаграми діяльності тощо, діаграма класів є найпопулярнішою діаграмою UML.

Приклад діаграми класів



Переваги діаграми класів.

- Будь-яку просту або складну модель даних можна проілюструвати за допомогою діаграми класів для отримання максимальної інформації.
- Схематику програми можна зрозуміти за допомогою неї.
- Будь-яка потреба в системі може бути візуалізована та передана по всьому бізнесу для конкретних дій.
- Будь-яка вимога щодо реалізації конкретного коду може бути виділена за допомогою діаграм і запрограмована до описаної структури.
- Опис, незалежний від реалізації, може бути наданий та переданий компонентам.

Недоліки діаграми класів.

- Діаграми класів часто можуть зайняти більше часу та підтримувати.
- Складна діаграма не допомагає розробникам програмного забезпечення в їх роботі.
- Можуть виникнути ситуації, коли розробники засмучуються через структуру діаграм класів.
- Розміщення конструкції може перешкоджати розробникам та компаніям.
- Як бачите, незважаючи на важливість діаграми класів у життєвому циклі розробки програмного забезпечення, вона, звичайно, не позбавлена недоліків і може ускладнити життя розробникам та компаніям.

Види діаграми класів.

Діаграма класів може бути поділена на три компоненти

- Верхній розділ, який складається з назви класу, і є обов'язковим компонентом.
- У середньому розділі описані якості класу та використовуються під час опису конкретного екземпляра класу.
- У нижньому розділі описано взаємодію класу з даними.

Діаграма класів (class diagram)

- На діаграмах класів зазвичай представлені такі елементи:
- Класи;
- Інтерфейси;
- Залежності, узагальнення та асоціації
- Також діаграми можуть включати в себе пакети або підсистеми; ті та інші групують елементи моделі у більш великі утворення. Іноді в діаграму класів потрібно додати примірники

Клас

- Клас в мові UML служить для позначення множини об'єктів, які мають однакову структуру, поведінку і відносини з об'єктами інших класів.
- Графічно клас зображується у вигляді прямокутника, який додатково може бути розділений горизонтальними лініями на розділи або секції. У цих розділах можуть зазначатися ім'я класу, атрибути (змінні) та операції (методи).



Клас

- **Ім'я класу** має бути унікальним в межах пакету, який описується деякою сукупністю діаграм класів або однією діаграмою.
- Ім'я вказується в самій верхній секції прямокутника, тому вона часто називається секцією імені класу.
- Ім'я класу записується по центрі секції імені напіжирним шрифтом і повинне починатися з великої літери.
- Для позначення імені абстрактного класу використовується курсив.
- У деяких випадках необхідно вказати, до якого пакета відноситься клас. Для цього використовується спеціальний символ розподільник – подвійна двокрапка – (::) Синтаксис рядка буде таким <Ім'я пакету>::<Ім'я класу>
- Наприклад Банк:: Рахунок

Атрибути класу

- **Ім'я атрибута** – рядок тексту, який використовується в якості ідентифікатора відповідного атрибута і тому повинний бути унікальним в межах даного класу.
- **Атрибути** класу або властивості записуються у другій зверху секції прямокутника класу. В UML кожному атрибуту класу відповідає окремий рядок тексту, який складається з квантора видимості атрибута і , можливо, початкового значення:
- <квантор видимості><ім'я атрибута>[кратність]: <тип атрибута> = <початкове значення>{рядок властивості}

Атрибути класу

- Квантор видимості може приймати одне із трьох можливих значень і відображається за допомогою відповідних спеціальних символів:
- «+» позначає атрибут з областю видимості типу загальнодоступний (public). Атрибут з цією областю видимості доступний з будь якого іншого класу пакету, в якому визначена діаграма;
- «#» атрибут із зоною видимості типу захищений (protected). Недоступний для всіх класів, за винятком підкласів даного класу;
- «-» атрибут із зоною видимості типу закритий «private». Недоступний для всіх класів без винятків;
- «~» видимість для елементів того ж простору імен (пакета) (package). Квантор видимості може бути опущений.

Операції (методи) класу

- У третій зверху секції прямокутника записуються операції або методи класу.
- Операція (operation) представляє собою деякий сервіс, який надає кожен екземпляр класу на певну вимогу.
- Кожній операції класу відповідає окремий рядок, який складається з квантору видимості операції, імені операції, виразу значення, що типу повертаються операцією і, можливо, рядок властивість операції:
- *<квантор видимості><ім'я методу>(список параметрів): <вираз типу значення, що повертається>{рядок властивості}*
- Ім'я операції – єдиний обов'язковий елемент, повинне починатися з маленької літри і, як правило, записуватися без пробілів

Відношення між класами

- Крім внутрішньої будови або структури класів на відповідній діаграмі вказуються відносини між класами.
- Кожне з цих відношень має власне графічне представлення на діаграмі, яке відображає взаємозв'язки між об'єктами відповідних класів.

залежності;
асоціації;
узагальнення;
реалізація.

	Залежність
	Асоціація
	Узагальнення
	Реалізація

Відношення залежності

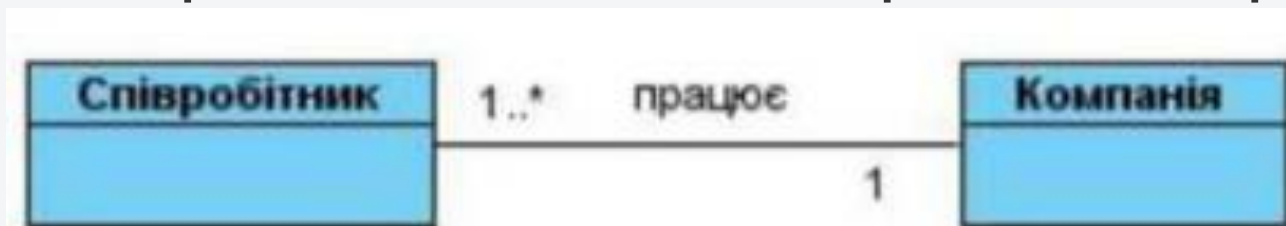
- Відношення залежності графічно зображуються пунктирною лінією між відповідними елементами зі стрілкою, направленою від класу-клієнту залежності до незалежного класу або класу-джерела
- Для відносин залежності зумовлені ключові слова, які позначають деякі спеціальні його види:
 - «access» - для позначення доступності відкритих атрибутів і операцій класу-джерела для класів-клієнтів;
 - «bind» - клас-клієнт може використовувати деякий шаблон для своєї подальшої параметризації;
 - «derive» - атрибути класу-клієнту можуть бути обчислені по атрибутах класу-джерела;
 - «import» - відкриті атрибути і операції класу-джерела стають частиною класу-клієнту, так, ніби вони були оголошені безпосередньо в ньому;
 - «refine» - вказує, що клас-клієнт служить уточненням клас-джерела в силу причин історичного характеру, коли з'являється додаткова інформація в ході роботи над проектом.

Відношення асоціації

- Відношення асоціації відповідає наявності деякого відношення між класами.
- Дане відношення позначається суцільною лінією з додатковими спеціальними символами, які характеризують окремі властивості конкретної асоціації.
- Ім'я асоціації є необов'язковим елементом її позначення.
- Якщо воно задане, то записується з великої букви поруч з лінією відповідної асоціації. Д
- ля асоціації може позначатися кількість екземплярів об'єктів кожного класу, які беруть участь у зв'язку
 - *(0 - якщо жодного, 1 - якщо один, * - якщо багато)*
- Можуть вказуватися мінімальна й максимальна кількість, наприклад, 0,1...* означає, що на відповідному кінці асоціації може не бути жодного екземпляра, бути один або багато

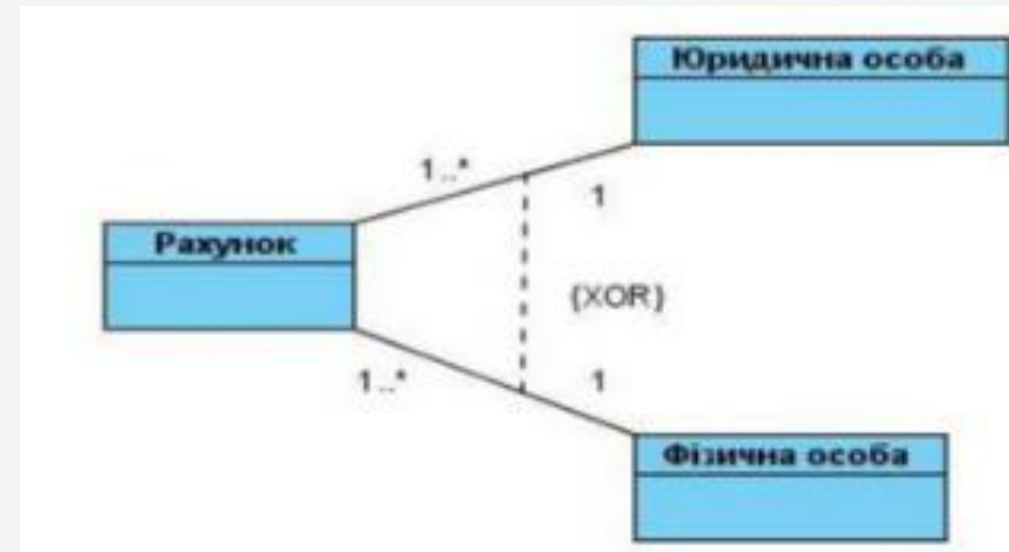
Відношення асоціації

- Бінарна асоціація – служить для зображення довільного відношення між двома класами. Вона зв'язує в точності два різних класи й може бути ненаправленим або направленим відношенням. Окремий випадок бінарної асоціації- рефлексійна асоціація, що зв'язує клас із самим собою. Ненаправлена асоціація зображується лінією без стрілки.
- Як простий приклад ненаправленої бінарної асоціації можна розглянути відношення між двома класами – класом Компанія та класом Співробітник. Вони зв'язані між собою бінарною асоціацією «працює». Для даного відношення визначений наступний порядок читання- співробітник працює в компанії.



Відношення асоціації

- Окремий випадок відносини асоціації - так звана виключає асоціація (Xor-association). Семантика даної асоціації вказує на те, що з кількох потенційно можливих варіантів даної асоціації в кожен момент часу може використовуватися тільки один. На діаграмі класів виключає асоціація зображується пунктирною лінією, що з'єднує дві і більше асоціації, Поруч з якою записується обмеження у формі рядка тексту в фігурних дужках: {xor}.



Відношення узагальнення.

- Узагальнення – Відносини між більше загальним поняттям (предком) і менш загальним поняттям (нащадком)
- Менш загальний елемент моделі повинен бути позгоджений з більш загальним елементом і може містити додаткову інформацію. Дане відношення використовується для подання ієрархічних взаємозв'язків між різними елементами мови UML, такими як пакети, класи, варіанти використання.
- **Наслідування (Inheritance)** - Спеціальний концептуальний механізм, за допомогою якого більш спеціальні елементи включають в себе структуру і поведінку більш загальних елементів. клас-нащадок має всі властивості і поведінкою класу-предка, а також має власні властивості і поведінку, які можуть бути відсутніми у класу-предка.

Відношення узагальнення.

- Батько, предок (Parent) - щодо узагальнення більш загальний елемент.
- Нащадок (Child) - спеціалізація одного з елементів відносини узагальнення, званого в цьому випадку батьком.
- На діаграмах відношення узагальнення позначається суцільною лінією з трикутною стрілкою на одному з кінців. Стрілка вказує на більш загальний клас (клас-предок), а її початок - на більш спеціальний клас (клас-нащадок)



Відношення Агрегація.

- Агрегація (Aggregation) - Спеціальна форма асоціації, яка служить для подання відносини типу "частина-ціле" між агрегатом (ціле) і його складовою частиною)
- Ставлення агрегації має місце між декількома класами в тому випадку, якщо один з класів є сутність, яка включає в себе в якості складових частин інші сутності.
- Графічно відношення агрегації зображується суцільною лінією, один з кінців якої являє собою не зафарбований усередині ромб. Цей ромб вказує на той клас, який являє собою "ціле" або клас-контейнер. Решта класи є його "частинами"



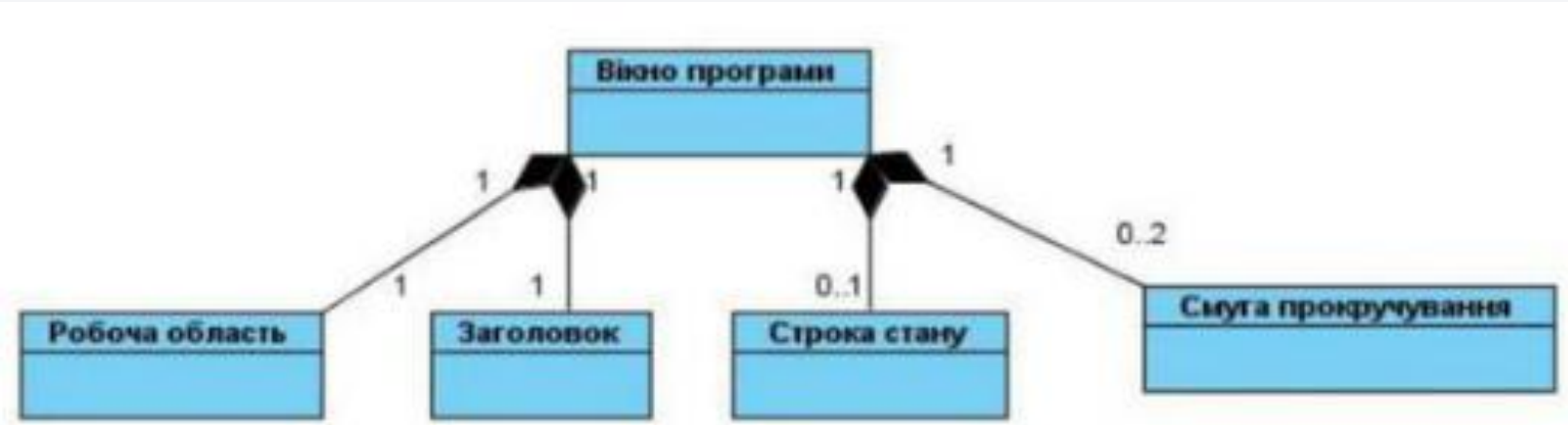
Відношення Агрегація.

- Як приклад відносини агрегації можна розглянути взаємозв'язок типу "частина-ціле", яка має місце між класом Системний блок персонального комп'ютера і його складовими частинами: Процесор, Материнська плата, Оперативна пам'ять, Жорсткий диск і Гнучкий диск. Використовуючи позначення мови UML, компонентний склад системного блоку можна представити у вигляді відповідної діаграми класів, яка в даному випадку ілюструє ставлення агрегації.



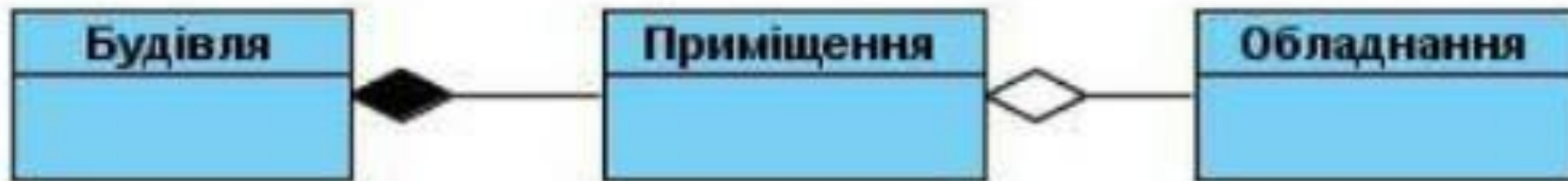
Відношення композиції

- Композиція (Composition) - Різновид відносини агрегації, при якій складові частини цілого мають такий же час життя, що й саме ціле. Ці частини знищуються разом зі знищенням цілого.
- Графічно відношення композиції зображується суцільною лінією, один з кінців якої являє собою зафарбований усередині ромб. Цей ромб вказує на той клас, який представляє собою клас-комполит. Решта класи є його "частинами"



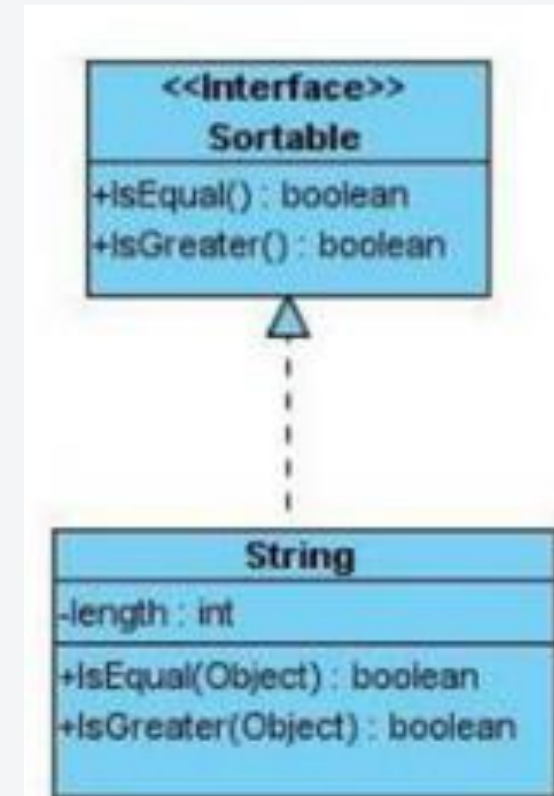
Відношення композиції

- **Зауваження:** Для виявлення типу зв'язку – агрегація чи композиція варто проаналізувати, що буде з класом-частиною після знищення класу контейнеру. Чи матиме він окремий сенс в рамках поставленої задачі? Так, наприклад, після знищення Будівлі, Приміщення припиняє своє існування, а от Обладнання матиме сенс.



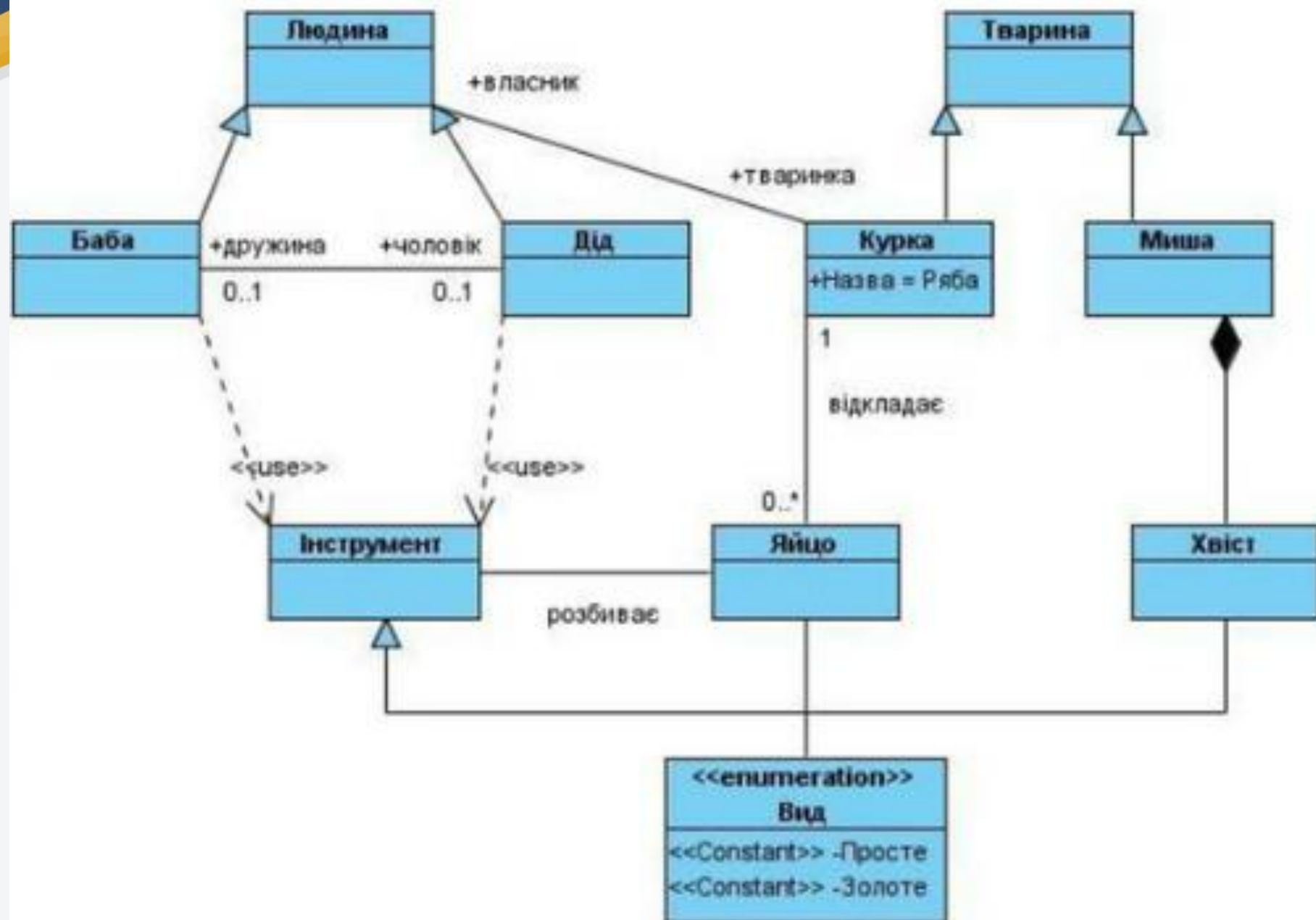
Відношення реалізації

- Реалізація (Realization) - це семантичне відношення між класифікаторами, при якому один класифікатор визначає сутність, а інший гарантує її виконання.
- Відносини реалізації в діаграмах класів зустрічаються між інтерфейсами і класами, що реалізують їх.
- Зображується відношення реалізації у вигляді пунктирної лінії з незафарбованою стрілкою, як щось середнє між відносинами узагальнення та залежності.



Приклад:

- Для наочності вищевикладеного матеріалу, розглянемо діаграму класів, яка ілюструє відому казку «Курка Ряба»
- У казці Дід (клас Дід) та Баба (клас Баба) є Людьми (класи Дід та Баба наслідують від класу Людина), та вони є власниками (відношення асоціації) Курки (клас Курка) на ім'я Ряба.
- В свою чергу, Курка Ряба є твариною (тобто клас Курка наслідує від класу Тварина).
- Курка може відкладати яйця (клас Яйце, відношення асоціації «відкладати»).
- Зауважимо, що яйце в казці фігурувало як золоте, так і просте, що відображується на діаграмі за допомогою класу <> Вид.
- Ще однією дійовою особою казки є Миша (клас Миша), яка також є Твариною та володіє Хвостом (клас Хвіст) як інструментом (клас Інструмент) для розбиття яйця (відношення асоціації «розбиває»).

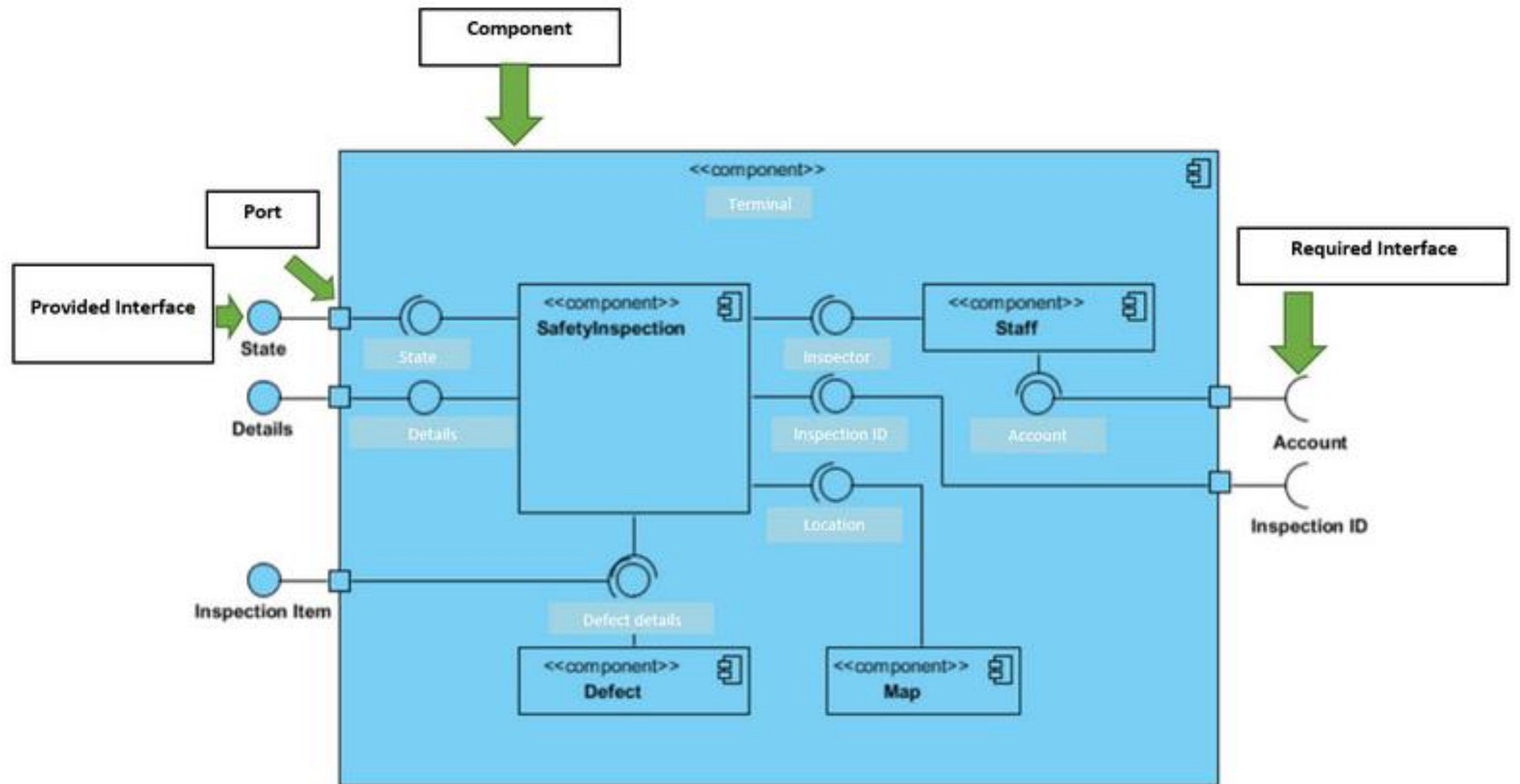


Діаграма компонентів

- Діаграма компонент (англ. Component diagram) — в UML, діаграма, на якій відображаються компоненти, залежності та зв'язки між ними.

Діаграма компонентів

- Діаграми компонентів UML надають концептуальну картину взаємодії між різними системами. Можуть бути присутні як аспекти логічного, так і фізичного моделювання. Крім того, компоненти автономні. Це модульний системний елемент в UML, який можна замінити на альтернативні. Вони містять конструкції будь-якої складності та є самодостатніми. Лише через інтерфейси вкладені частини взаємодіють з іншими компонентами. Крім того, компоненти мають свої інтерфейси, але вони також можуть отримати доступ до операцій і служб інших компонентів за допомогою їхніх інтерфейсів. На діаграмі компонентів інтерфейси також показують зв'язки та залежності в архітектурі програмного забезпечення.



Пояснення

- ♦ Дані, включно з обліковим записом та ідентифікатором перевірки, надходять у компонент через порт з правого боку. Потім він перетворюється у формат, зрозумілий внутрішнім компонентам. Інтерфейси праворуч називаються необхідними інтерфейсами. Вони відображають послуги, необхідні для виконання компонентом своїх функцій.
- ♦ Потім дані проходять через численні з'єднання до кількох інших компонентів перед тим, як вивести їх на порти ліворуч. Інтерфейси ліворуч називаються наданими інтерфейсами та відображають послуги, які надаватиме представлений компонент.
- ♦ Велика квадратна форма може бути системою. Крім того, це може бути підсистема або компонент системи, який оточує внутрішні компоненти.

Символи діаграми компонентів UML

Примітка Символ

Це дає програмістам можливість додавати мета-аналіз до діаграми компонентів.



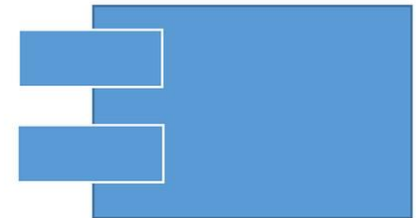
Символ вузла

Він представляє елементи, які мають вищий рівень, ніж компоненти, наприклад апаратне чи програмне забезпечення.



Символ компонента

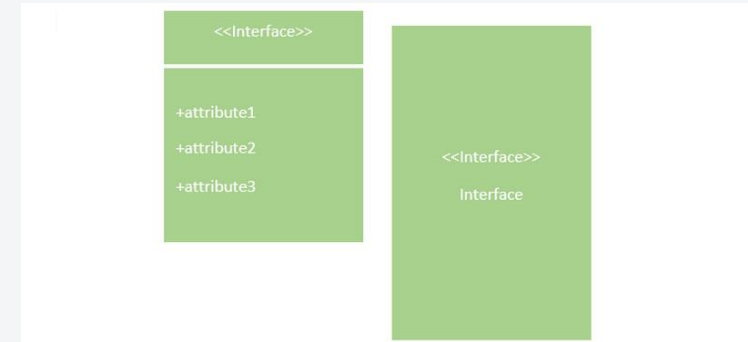
Цей символ є річчю, необхідною для виконання стереотипного завдання. Компонент взаємодіє з іншими компонентами та пропонує та споживає поведінку через інтерфейси. Розглядайте компоненти як певний тип класу. Компонент представлений в UML 1.0 у вигляді прямокутного блоку з двома меншими прямокутниками, що виступають з боків. Компонент в UML 2.0 представлений у вигляді прямокутного блоку з крихітним представленням форми діаграми попереднього компонента.



Символи діаграми компонентів UML

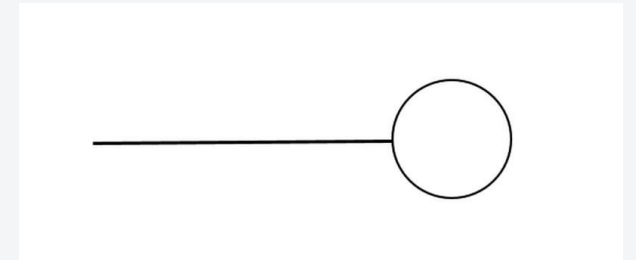
Символ інтерфейсу

Він відображає будь-які вхідні дані або матеріали, які надсилає або отримує компонент. Текстові примітки або символи, як-от форми льодяника, розетки та кулі та розетки, можуть позначати інтерфейси.



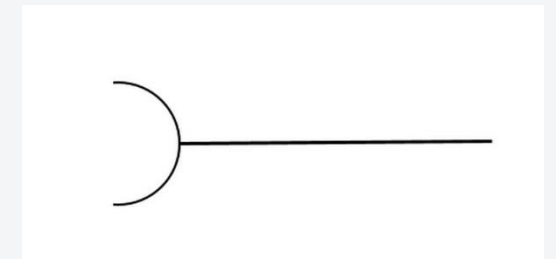
Необхідний інтерфейс

Він отримує послуги, функції або дані ззовні. Його ще називають льодяником.



Наданий інтерфейс

Це символ для визначення інтерфейсів, які надають функції, дані або послуги ззовні. Півколо називається розеткою.



Символи діаграми компонентів UML

Символ порту

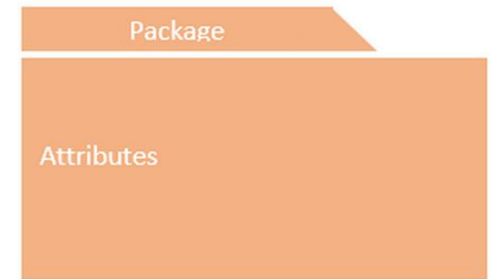
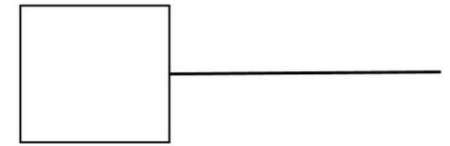
Точка взаємодії компонента з середовищем позначається окремо. Крихітний квадрат служить символом портів.

Пакет символ

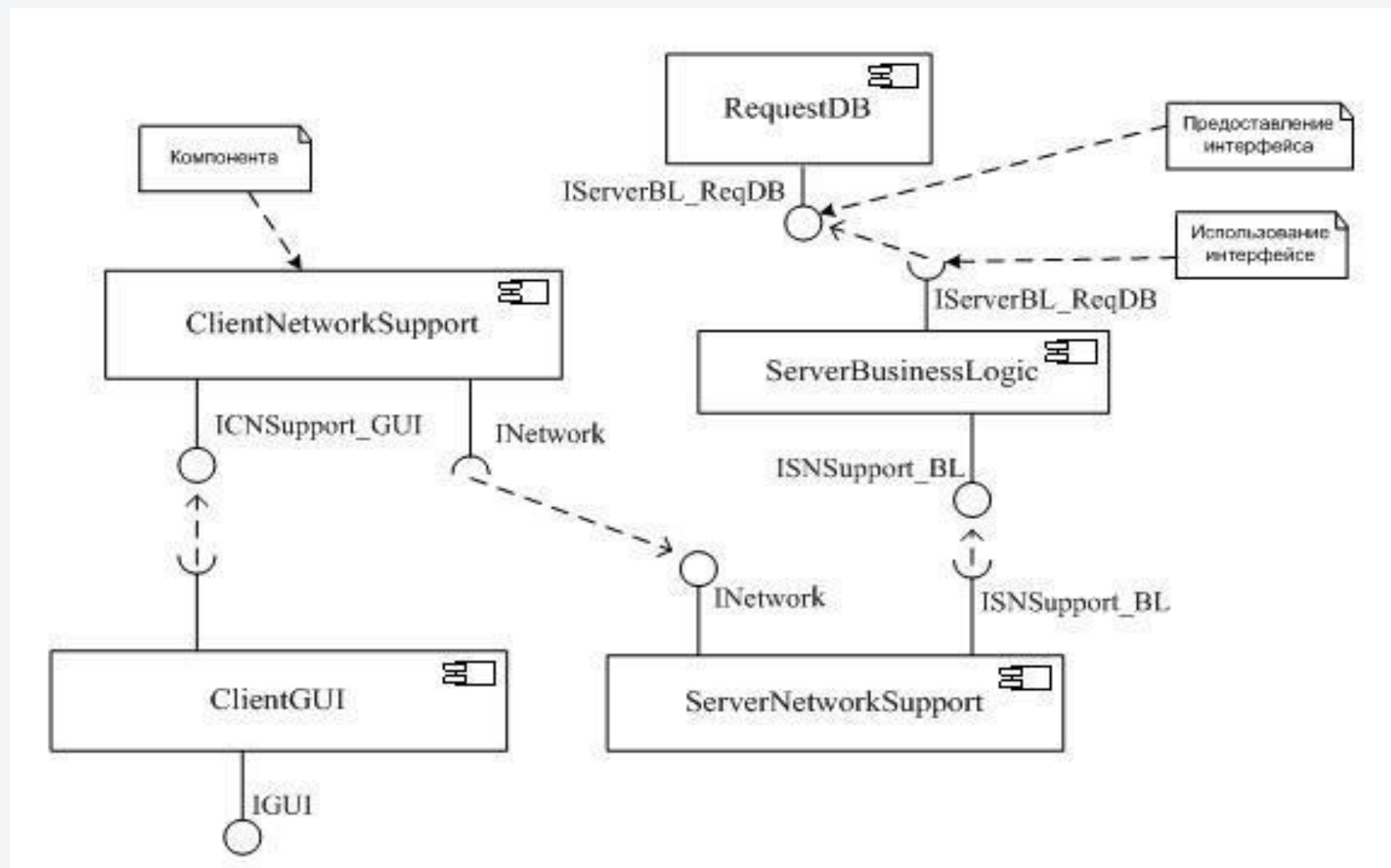
Цей символ об'єднує різні елементи певної системи в групу. Він включає інтерфейси та класи компонентів.

Символ залежності

Це демонструє взаємозалежність різних компонентів системи. Пунктирні лінії, що з'єднують один компонент з іншим, представляють залежності.



Приклад діаграми компонентів

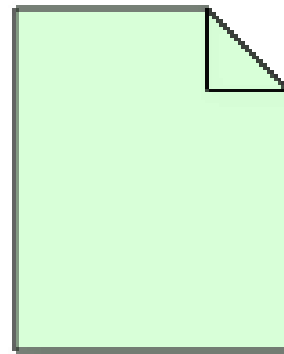
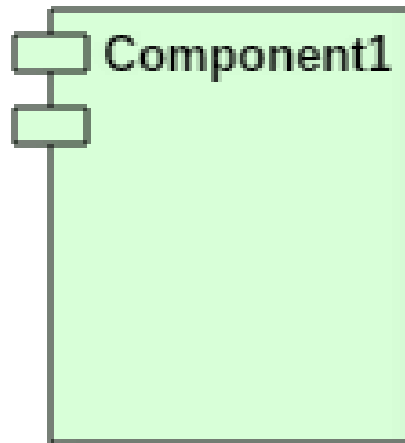


Діаграма розгортання

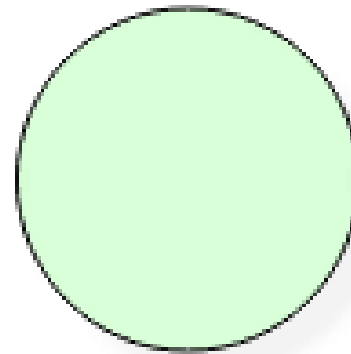
- Діаграма розгортання (англ. deployment diagram) — діаграма в UML, на якій відображаються обчислювальні вузли під час роботи програми, компоненти, та об'єкти, що виконуються на цих вузлах. Компоненти відповідають представленню робочих екземплярів одиниць коду.

Схема розгортання

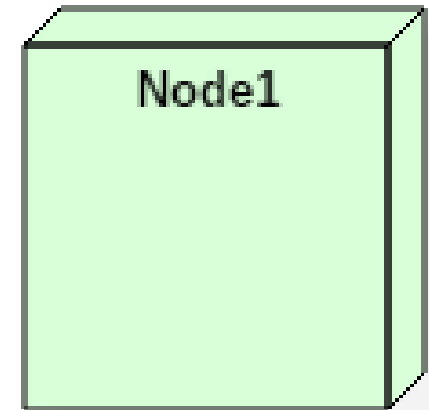
Символ і позначення



Artifact1



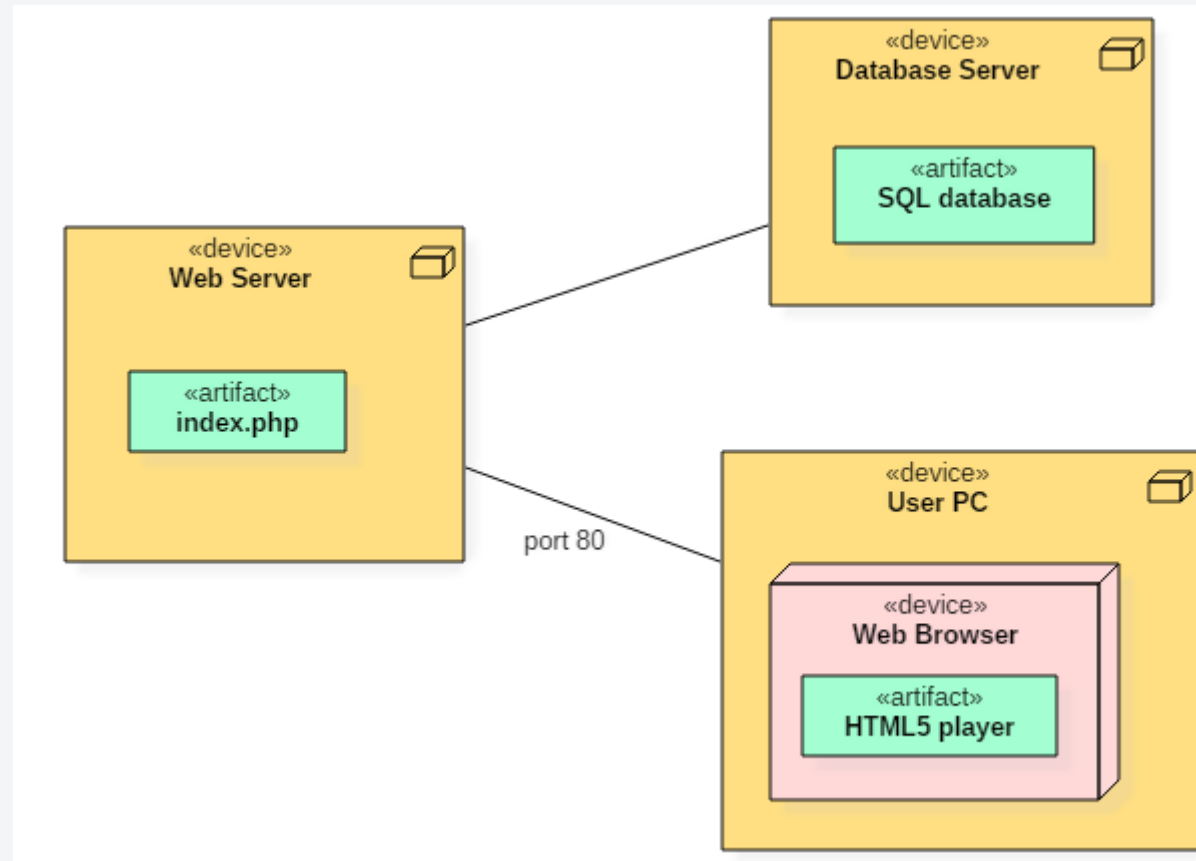
Interface1



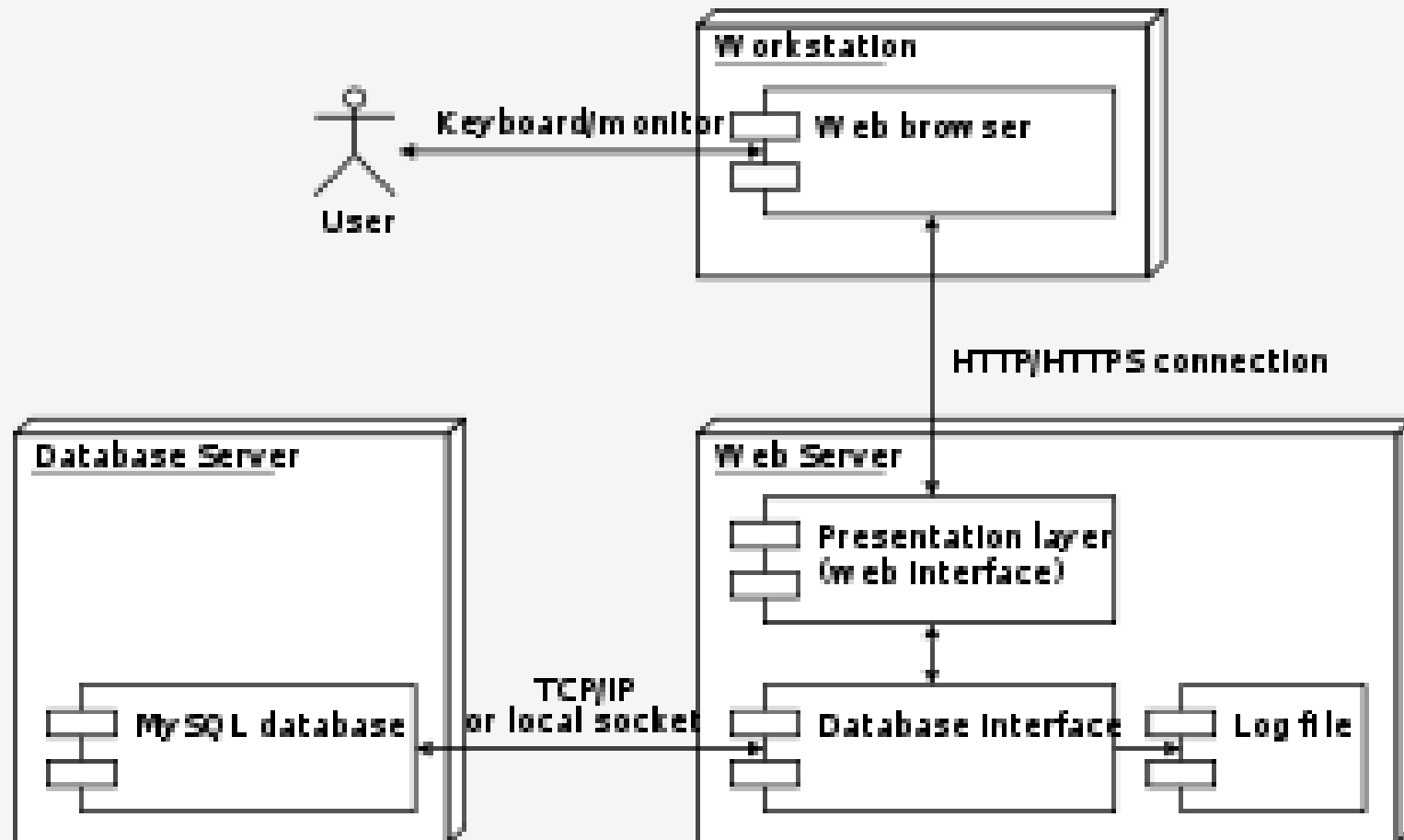
Node1

1. Вузол
2. Компонент
3. Артефакт
4. Інтерфейс

Приклад діаграми розгортання



Приклад діаграми розгортання



Популярні програми для створення діаграм.

- [Diagrams.net](#) — зручний сервіс для створення блок-схем, UML-діаграм, моделей бізнес-процесів онлайн. Сумісний з більшістю популярних інструментів, включаючи Google Docs, Git, Dropbox, One. Drive та інші.
- [Dbdiagram.io](#) — додаток для побудови діаграм зв'язків для баз даних. Хороший інструмент для розробників і аналітиків.
- [Google Drawings](#) — безкоштовний інструмент для створення блок-схем і діаграм в складі Google Drive (менш зручний порівняно з [diagrams.net](#))